



Digital Filter 3

デジタルフィルタ 3

FIR および IIR 方式のデジタルフィルタの実装方法を理解し, FFT により周波数特性を観察してみよう

Understand how to **implement FIR and IIR digital filters** and observe their **frequency** characteristics using **FFT**

教科書の該当範囲: 第4章, 第6章

Textbook Coverage: Chapter 4, 6

デジタルフィルタの具体例として、残響・反響効果の付与に利用されるリバーブフィルタ "reverb filter" を採り上げる。

As a specific example of a digital filter, we will look at a **reverb filter**, which is used to add **reverberation** and **reflection effects**.

効果による分類: Classification by effect → (reverb and echo)

残響 "reverb" or "reverberation": 音源物体自身による持続振動音。揺れ残り。物体の構造等にもよるが、遅延は小さめ

Reverberation: A sustained **vibration** caused by the **sound source itself**. The residual vibration. The **delay is small**, depending on the structure of the object

反響 "echo": 周囲物体(壁, 床, 等)による反射音。物体の構造等にもよるが、遅延は大きめ(距離に依存)

Echo: **Sound reflected** by surrounding objects (walls, floors, etc.). The **delay is large** (depending on distance), depending on the structure of the object.

処理方式による分類:

Classification by processing method → 2 (FIR and IIR)

FIR 方式: タップ数 J は非常に大 (持続時間に依存), フィルタ係数 b_m は小さめ

FIR method: The number of taps J is very large (depends on the duration), and the filter coefficients b_m are small

大量の係数 × 入力 が加算されて, 出力となる. 音割れ防止策として, 係数の和を $\sum_m |b_m| < 1$ とかに

A large number of coefficients x inputs are added together to produce the output. To prevent distortion, the sum of the coefficients is set to $\sum_m |b_m| < 1$

IIR 方式：帰還部のタップ数 I は小，フィルタ係数 a_m は大きめ（とは云え 1.0 未満，持続時間に依存）

IIR method: The number of taps in the feedback section I is small, and the filter coefficient a_m is large (? , but less than 1.0, depending on the duration)

音割れ防止策として，入力側のフィルタ係数を $b_0 = 0.5$ 程度に

To prevent distortion, set the input filter coefficient to about $b_0 = 0.5$

反響の遅延の算定： Calculating echo delay

→ 距離と音速の関係式： $2r = vt$ Relationship between distance and speed of sound: $2r = vt$

r ：音源から反射物体までの距離 r : Distance from the sound source to the reflecting object

v ：音速，常温・常圧で約 340 [m/s] v : Speed of sound, approximately 340 [m/s] at standard temperature and pressure

t ：反射音の往復時間 t : Round-trip time of the reflected sound

遅延と標本化周波数の関係式: $d = f_s t$ Relationship between delay and sampling frequency: $d = f_s t$

d : 遅延 (標本数) d : Delay (number of samples)

f_s : 標本化周波数. 標準的には 48 [kHz] f_s : Sampling frequency. Standard value is 48 [kHz]

遅延と距離の関係式:

$$d = \frac{2f_s}{v} r \simeq 282 r \simeq 300 r$$

Relationship between delay and distance

例えば、距離が 10 [m] としても、反射音の遅延は 3,000 標本にもなる。より大規模なコンサートホールだと、どうなる？ さらに、反射音の残響の持続時間分の遅延と乗算も必要。これを FIR で実装する場合、フィルタ係数のメモリ容量・乗算回数について、かなり高コストになりそうですね

For example, even if the distance is 10 [m], the delay of the reflected sound will be 3,000 samples. What will happen in a larger concert hall? In addition, delay and multiplication for the duration of the reverberation of the reflected sound are also required. If this is implemented with FIR, it seems that the memory capacity of the filter coefficients and the number of multiplications will be quite high cost.



作業用ディレクトリ: 以前の通り ~/sp/pcm/ 等 Working directory: ~/sp/pcm/ etc. as before

改造元ソースファイル: 以前に作成したどれか Source file to be modified: one created previously

サンプルプログラム(ソースコード断片の画像): iir.c

Sample program (image of source code fragment): iir.c

```
int main(void)
{
    // パラメータ
    double b0 = 0.5;      // 入力側のフィルタ係数 (音割れ防止の調整用)

    double a1 = -0.2;     // 残響成分のフィルタ係数
    int d1 = 1;           // 残響成分の遅延

    double a2 = 0.7;      // 反響成分のフィルタ係数
    int d2 = 282*10;      // 反響成分の遅延
                        //  $2 * r * f_s / v \approx 282 * r$ 
                        // r: 反射物体までの距離 [m],  $f_s=48$ [kHz],  $v=340$ [m/s]

    double dt = 1.0;      // 残響用の延長収録時間 [s]
```



```
// 信号の入力
Wav      *p0 = pcmRead(stdin);
if (p0 == NULL) fatal(1, "p0: Read失敗\n");
int       bit = p0->fmt.bit;      // 量子化深度 (共通)
int       fs = p0->fmt.fs;        // 標本化周波数 [kHz] (共通)
int       len0 = p0->len;         // 標本数 (入力)

// 入力信号のコピー & 延長
int       dn = ceil(dt*fs);       // 標本数の延長量
int       len = len0 + dn;        // 延長後の標本数
Wav      *p1 = pcmInit(bit, 1, fs, len);
// 残響追加用に標本数増量 (手抜きバッファオーバーラン対策)
// チャンネル数 1 (やる気ある人はステレオにも対応しては?)
if (p1 == NULL) fatal(1, "p1: Init失敗\n");
double   *x = p1->val[0];        // 入力標本値列へのポインタ
memcpy(x, p0->val[0], sizeof(double)*len0);
// コピーの標本数はlenではなくlen0だぜー? わかるかな?
// バッファ末尾はゼロ埋めのままに
pcmFin(p0);    // 入力信号の始末
```



```
// 出力信号の準備
Wav      *p2 = pcmInit(bit, 1, fs, len);
if (p2 == NULL) fatal(1, "p2: Init失敗\n");
double *y = p2->val[0];          // 出力標本値列へのポインタ

// 標本値取得マクロ（手抜きバッファオーバーラン対策）
#define X(n)    (((n) < 0) ? 0.0 : x[n])    // x[n]
#define Y(n)    (((n) < 0) ? 0.0 : y[n])    // y[n]

// フィルタリング & 出力
for (int n = 0; n < len; n++) {
    y[n] = b0*X(n) - a1*Y(n-d1) - a2*Y(n-d2);
    /* 標本値 x[n] や y[n-d] へのアクセスでは、
       バッファオーバーランの危険がアブない。←重複表現
       n-d < 0 → マクロX(), Y()で回避してね。
       n >= len0 → 安心してください、延長してますよ。バッファ。 */
}
pcmWrite(stdout, p2);

// 後始末
pcmFin(p1);
pcmFin(p2);
return (0);
```



実験(聴き比べ):

Experiment : Listening Comparison

```
$ cc iir.c -std=c99 -L. -lpcm -lm -o iir
$ paplay /usr/share/sounds/alsa/Front_Center.wav # 原音
$ cat /usr/share/sounds/alsa/Front_Center.wav | ./iir | paplay # 加工音
```

→ 違いますよね？ 響いた？ Isn't that right? Did it resonate with you?

実験(効果の調整): 各自でパラメータ(遅延およびフィルタ係数の数値)の大小・符号を様々に変更し, 効果を比較してみよう

Experiment (adjusting the effect): Try changing the magnitude and sign of the parameters (delay and filter coefficients) in various ways and compare the effects

→ 割れや不安定化(発散・発振)しないよう注意 Be careful not to cause cracking or instability (divergence/oscillation)

改造: 余裕あれば, 帰還経路を増量したり, フィルタ構成を直列・並列に多段化してみてもは？

Modification: If you have the time, why not try increasing the number of feedback paths or making the filter configuration multi-stage in series or parallel?

サンプルプログラム: [ip.c](#) →

Sample Program : ip.c

```
// インパルス生成
// Ver.20240126
// コンパイル: $ cc ip.c -std=c99 -lpcm -o ip
// 実行方法: $ ./ip [標本数]
// 実行例: $ ./ip | paplay

#include <stdio.h>
#include <stdlib.h>
#include "pcm.h"

int main(int argc, char *argv[])
{
    int len = 512;
    if (argc > 1) len = atoi(argv[1]);
    Wav *p = pcmInit(16, 1, 48000, len);
    if (p == NULL) return (1);

    p->val[0][0] = 1.0;
    pcmWrite(stdout, p);

    pcmInfo(stderr, p);
    pcmFin(p);
    return (0);
}
```

インパルスおよびインパルス応答の生成:

Generating impulses and impulse responses

```
$ cc ip.c -std=c99 -L. -lpcm -o ip
$ ./ip > ip.wav # インパルスの生成
$ paplay ip.wav # 「ブツッ」
$ cat ip.wav | ./iir > ir.wav # インパルス応答の生成
$ paplay ir.wav
```

インパルス応答の標本値列は, FIR 用のフィルタ係数列 b_m ですよ

The sample value sequence of the impulse response is the filter coefficient sequence for FIR, b_m

FFT の準備: 以前作成した dft.c を元にして, main() 内の処理時間測定や DFT(), IDFT(), 等の呼出を削除し, FFT() の呼出だけに変更

Preparation for FFT: Based on the previously created dft.c, delete the processing time measurement in main() and calls to DFT(), IDFT(), etc., and change it to only FFT()

→ 以下の実験は, DFT() でもできますが, FFT() の方が好都合

The following experiment can also be done with DFT(), but FFT() is more convenient

FFT による伝達関数の算出:

Calculating the transfer function using FFT

```
$ ./dft ir.wav 標本数 始点
# 標本数と始点には, 残響・反響が現れている区間を指定.
# パラメータから推計したり, 試行錯誤で決定.
$ ./dw.plot wf.txt # 波形表示
$ ./ds.plot fft.txt # スペクトル表示
```

インパルス応答のスペクトルは, 伝達関数 H_k です

The spectrum of the impulse response is the transfer function H_k

FIR 方式では、インパルス応答が利用可能であれば、既存の複数のフィルタをモノマネして容易に再現できる

In the FIR method, if the impulse response is available, it is easy to reproduce multiple existing filters by imitating them.

→ 実装はひとつでも、特性は切り替え可能

Even if there is only **one implementation**, the characteristics can be switched.

サンプルプログラム(断片): fir.c(iir.c を元に改造)

Sample program (fragment): fir.c (modified from iir.c)

```
int main(int argc, char *argv[])
{
    // フィルタ係数 (インパルス応答) の入力
    char *fn = "ir.wav";
    if (argc > 1) fn = argv[1];
    Wav *ir = pcmLoad(fn);
    if (ir == NULL) fatal(1, "ir: Read失敗\n");
    double *b = ir->val[0];
    int nb = ir->len;
    ...

    // 入力信号のコピー & 延長
    int dn = nb; // 標本数の延長量
    int len = len0 + dn; // 延長後の標本数
    ...

    // フィルタリング & 出力
    for (int n = 0; n < len; n++) {
        // 畳み込み
        y[n] = 0;
        for (int m = 0; m < nb; m++) {
            y[n] += b[m]*X(n-m);
        }
    }
    ...
}
```



実験 (聴き比べ) :

Experiment (listening comparison)

```
$ cc fir.c -std=c99 -L. -lpcm -o ip
$ cat /usr/share/sounds/alsa/Front_Center.wav | ./fir ir.wav | paplay # フィルタリングに時間がかかります
$ cat /usr/share/sounds/alsa/Front_Center.wav | ./iir | paplay
```

→ 理想的には同じ音のハズ

Ideally, the sound should be the same

→ FIR では、計算量が多いので、レイテンシが問題. インパルス応答の末尾の小信号区間を切除して、タップ数を減らすしかないでしょう

FIR requires a lot of calculations, so latency is an issue. The only solution is to cut off the small signal section at the end of the impulse response and reduce the number of taps